



THEEM COLLEGE OF ARTS, COMMERCE & SCIENCE



Name : Vikas Gupta



Roll No. : 10



Sub : IKS

TOPIC

Meru-Prastāra
as Pascal Triangle and
Dynamic Programming Model



INDEX

SR. No.	Section	
1.	IKS Concept as CS Concept	2,3
2.	Historical Background & Mathematical Foundations of Meru-Prastāra	3,4
3.	Problem Statement & Detailed Objectives	4,5
4.	Comprehensive Conceptual Mapping Table	5
5.	Algorithm Specification & Mathematical Pseudocode	6
6.	Optimized Working Python Code	6,7,8
7.	Minimum 15 Test Cases and Results	8,9,10
8.	Rigorous Complexity Analysis & Hardware Limitations	10,11
9.	Applications in Modern Computer Science & Combinatorics	11
10.	Conclusion & Future Scope	11

IKS Implementation Work

Topic

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

1.Title: IKS Concept as CS Concept

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

❖ Introduction to Meru-Prastāra

Meru-Prastāra is an ancient Indian mathematical and poetic concept deeply associated with the systematic study of combinations, patterns, and structural arrangements. The term literally translates to "Staircase of Mount Meru" (where Mount Meru represents the sacred cosmic mountain in ancient cosmology, symbolizing a tiered, ascending structure). Historically, this concept was pioneered by ancient Indian scholars such as Acharya Pingala in his treatise *Chhandah-shastra* (around the 3rd to 2nd century BCE), primarily for analyzing meters (chhandas) in Sanskrit poetry.

Pingala and subsequent mathematicians like Halayudha (who explicitly commented on and expanded the Meru-Prastāra matrix in the 10th century CE) developed methods for systematically representing combinations of short (*laghu*) and long (*guru*) syllables. This demonstrates that ancient Indian

mathematical thinking was heavily intertwined with linguistics, phonetics, poetry, and algorithmic enumeration.

The Bridge to Pascal's Triangle

The structural layout of Meru-Prastāra mirrors what Western mathematics later popularized as Pascal's Triangle (named after the French mathematician Blaise Pascal, centuries later). Pascal's Triangle is a geometric arrangement of numbers structured in a triangular form where each row begins and ends with 1, and every internal element is derived by summing the two values directly positioned above it.

Mathematically, the entries of this triangle represent binomial coefficients, denoted as $C(n, k)$ or $\binom{n}{k}$, which calculate the number of ways to choose k elements from a set of n elements without regard to order:

$$C(n, k) = \frac{n!}{k! (n - k)!}$$

Intersection with Computer Science

From a Computer Science perspective, Meru-Prastāra serves as a foundational bridge to understanding recursive thinking, array data structures, and optimization paradigms like **Dynamic Programming**. Rather than recalculating combinations using expensive factorial functions for every coordinate (which suffers from integer overflow and redundant computation), the triangular dependency allows us to solve subproblems sequentially and store them efficiently.

2. Historical Background & Mathematical Foundations of Meru-Prastāra

The Prosodic Origin

In ancient India, poetic meters were classified based on the number of syllables per line and the permutation of short (*laghu*) and long (*guru*) syllables. To find how many meters have a specific number of long and short syllables, scholars required a combinatorial tool. Meru-Prastāra provided a visual grid representation to compute these combinations instantly without complex multiplication.

Recurrence Relation Formulation

The mathematical recurrence governing Meru-Prastāra states that any interior coefficient is the sum of its upper-left and upper-right predecessors. Formally defined:

$$P(n, k) = P(n - 1, k - 1) + P(n - 1, k)$$

Subject to the boundary conditions:

- $P(n, 0) = 1$
- $P(n, n) = 1$

This recurrence relation eliminates the need for independent combinatorial evaluations, transforming an exponential problem into a polynomial-time lookup-and-sum operation.

3. Problem Statement & Detailed Objectives

Problem Definition

To design, implement, and analyze a software model that computes and generates **Meru-Prastāra (Pascal's Triangle)** using a **Bottom-Up Dynamic Programming (Tabulation)** approach. The program must systematically construct rows up to a user-defined limit n , ensuring that previously computed rows serve as the foundation for succeeding rows.

Project Objectives

- 1. Bridge ancient Indian mathematical heritage (*Indian Knowledge Systems*) with modern algorithmic paradigms.
- 2. Implement efficient memory management using Python lists/arrays.
- 3. Validate correctness through comprehensive edge-case testing (minimum 10+ test vectors).
- 4. Evaluate space and time complexities to understand resource scaling.

4. Comprehensive Conceptual Mapping Table

Meru-Prastāra / IKS Domain	Modern Computer Science Domain	Implementation Manifestation
Meru-Prastāra Grid	2D Data Structure / List of Lists	triangle = [] storing dynamic rows
Boundary Elements (1s)	Boundary Conditions	Base cases where row[0] = 1 and row[i] = 1
Internal Summation Pattern	Recurrence Relation	row[j] = triangle[i-1][j-1] + triangle[i-1][j]
Sequential Dependency	Dynamic Programming (Tabulation)	Reusing pre-calculated rows from memory
Syllable Combinations	Combinatorics & Binomial Coefficients	Calculating item selection patterns

5. Algorithm Specification & Mathematical Pseudocode

Algorithm: Tabulated Meru-Prastāra Generation

START

1. Read/Receive integer 'n' representing the total rows to generate.
2. Initialize an empty list: triangle = []
3. FOR i from 0 to n-1 DO:
 - a. Create a temporary row list of size (i + 1) initialized with zeros or ones.
 - b. Set boundary elements: row[0] = 1 and row[i] = 1.
 - c. FOR j from 1 to i-1 DO:
 - Fetch preceding row values from triangle[i-1].
 - Compute: row[j] = triangle[i-1][j-1] + triangle[i-1][j].
 - d. Append the constructed row into triangle.
4. RETURN triangle.
5. Display each row formatted structurally.

END

6.Optimized Working Python Code

```
def generate_meru_prastara(n):
```

```
    """
```

```
    Generates Meru-Prastara (Pascal's Triangle) using  
    Bottom-Up Dynamic Programming (Tabulation).
```

```
    """
```

```
if n <= 0:
```

```
    return []
```

```
triangle = []
```

```
for i in range(n):
```

```
    # Create a row of length i+1 filled with 1s
```

```
    row = [1] * (i + 1)
```

```
    # Calculate intermediate values using the previous row
```

```
    for j in range(1, i):
```

```
        row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j]
```

```
    triangle.append(row)
```

```
return triangle
```

```
def display_triangle(triangle):
```

```
    """Utility function to print the triangle in a readable format."""
```

```
    print("\nMeru-Prastara (Pascal's Triangle) Output:")
```

```
    for idx, row in enumerate(triangle):
```

```
        print(f"Row {idx:2d}: {row}")
```



```

if __name__ == "__main__":

    try:

        n_rows = int(input("Enter number of rows to generate: "))

        result = generate_meru_prastara(n_rows)

        display_triangle(result)

    except ValueError:

        print("Please enter a valid integer.")

```

7. Minimum 15 Test Cases and Results

To guarantee algorithmic reliability, the implementation was tested against various scalar inputs, boundary parameters, and large scales:

Test ID	Input (n)	Expected Core Output Structure	Validation Status
TC-01	0	[] (Empty structure)	Pass
TC-02	1	[[1]]	Pass
TC-03	2	[[1], [1, 1]]	Pass
TC-04	3	[[1], [1, 1], [1, 2, 1]]	Pass

Test ID	Input (n)	Expected Core Output Structure	Validation Status
TC-05	4	Last row: [1, 3, 3, 1]	Pass
TC-06	5	Last row: [1, 4, 6, 4, 1]	Pass
TC-07	6	Last row: [1, 5, 10, 10, 5, 1]	Pass
TC-08	7	Last row: [1, 6, 15, 20, 15, 6, 1]	Pass
TC-09	8	Last row: [1, 7, 21, 35, 35, 21, 7, 1]	Pass
TC-10	9	Last row: [1, 8, 28, 56, 70, 56, 28, 8, 1]	Pass
TC-11	10	Last row: [1, 9, 36, 84, 126, 126, 84, 36, 9, 1]	Pass
TC-12	12	Computes 78 total elements smoothly	Pass
TC-13	15	Computes 120 total elements smoothly	Pass

Test ID	Input (n)	Expected Core Output Structure	Validation Status
TC-14	20	Computes 210 total elements smoothly	Pass
TC-15	25	Computes 325 total elements smoothly	Pass

8. Rigorous Complexity Analysis & Hardware

Limitations

Time Complexity

The total number of additions performed to construct n rows corresponds to the summation of natural numbers:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Therefore, the Time Complexity is $O(n^2)$. Execution time scales quadratically relative to the row inputs.

Space Complexity

The complete triangle retains approximately $\frac{n^2}{2}$ integer nodes in system RAM. Thus, the Space Complexity is $O(n^2)$.

Practical Limitations

1. **Memory Ceiling:** Storing thousands of rows causes memory consumption to escalate rapidly due to list overheads in Python.

2. Integer Scaling: For massive values of n , coefficients grow exponentially, though Python handles arbitrarily large integers natively via its multi-precision integer backend.
3. Scope Constraints: The implementation highlights computational representation and leaves out manual poetic prosody applications.

9. Applications in Modern Computer Science & Combinatorics

- > **Binomial Expansions:** Quick evaluation of polynomial coefficients in algebra.
- > **Probability & Statistics:** Calculating combinations for binomial distributions and coin-toss outcomes.
- > **Pathfinding & Grid Combinatorics:** Determining route counts on grid-based systems.

10. Conclusion & Future Scope

This project successfully demonstrates the deep structural alignment between the ancient Indian mathematical traditions of Meru-Prastāra and modern computational Dynamic Programming. By treating past rows as reusable structural states, we optimized computation and built a clean, scalable Python system.

Future Scope: This model can be extended into 3D space (Pascal's Pyramid) or analyzed for parallel processing architectures to optimize multi-threaded matrix execution.